

# **Sri Lanka Institute of Information Technology**



**SE3040 – Application Frameworks  
Assignment 02**

## Contents

|                                    |   |
|------------------------------------|---|
| 1. Introduction .....              | 3 |
| 2. Technologies Used .....         | 3 |
| 3. Functional Requirements .....   | 3 |
| 4. System Architecture .....       | 4 |
| 5. Features & Functionality .....  | 4 |
| 6. API Integration Details .....   | 4 |
| 7. User Interface & UX .....       | 5 |
| 8. Session Management.....         | 5 |
| 9. Version Control.....            | 5 |
| 10. Deployment Strategy .....      | 6 |
| 11. Testing Plan .....             | 6 |
| 12. Challenges and Solutions ..... | 6 |
| 13. Conclusion .....               | 7 |

## 1. Introduction

This web application, "GeoNeo," is designed to serve as an intuitive and engaging platform for users who are curious about the world, planning to travel, or interested in learning about global geography. Whether you're a student studying countries and capitals, a traveler researching your next destination, or simply someone fascinated by global cultures, this website offers a user-friendly solution.

With categorized data views and powerful filtering tools, users can explore countries based on specific continents (regions), spoken languages, or by directly searching by name. The application fetches real-time data using the REST Countries API and dynamically displays country details, including the name, population, capital, languages, and national flag. The modern interface and responsive design ensure smooth navigation across devices, while the search and filter functionalities help users quickly find the information they need.

By incorporating features like detailed country cards and optional user login for session-specific features, this app aims to be a helpful digital companion for both educational and practical use cases. It enhances user engagement with clean visuals, interactive elements, and a smooth browsing experience.

## 2. Technologies Used

- **Frontend Framework:** React (Functional Components with Hooks)
- **Programming Language:** JavaScript
- **CSS Framework:** Tailwind CSS
- **API:** REST Countries API (<https://restcountries.com>)
- **Version Control:** Git & GitHub
- **Hosting Platform:** Vercel (Free tier)
- **Session Management:** LocalStorage (with backend using Node.js/Express)

## 3. Functional Requirements

- Display country data including:
  - Name
  - Capital
  - Region

- Population
  - Languages
  - Flag
- Enable users to:
  - Search for countries by name
  - Filter countries by region
  - Filter countries by language (optional)
  - Click on a country for more detailed information
- (Optional) Implement login/logout with persistent user sessions

## 4. System Architecture

- **React Components:** App, Header, CountryList, CountryCard, CountryDetails, SearchBar, FilterDropdown, LoginForm (optional)
- **Data Flow:** REST API → Axios/Fetch → State (useState/useEffect) → Components
- **State Management:** React Hooks and Context API (optional)
- **Authentication :** JWT-based login with backend API

## 5. Features & Functionality

- **Search Bar:** Dynamic filtering as the user types a country name.
- **Continent/Region Dropdown:** Real-time filter of countries by selected region.
- **Country Cards:** Each displays brief country info and can be clicked for more details.
- **Detailed View:** Full country details on click (optional but implemented).
- **Session Management:** Local storage to remember user session or preferences.
- **Responsive Design:** Fully mobile and desktop responsive using Tailwind CSS.

## 6. API Integration Details

Endpoints used from the REST Countries API:

- GET /all – Fetch all countries (used for initial data load)

- GET /name/{name} – Search countries by name
- GET /region/{region} – Filter countries by region
- GET /alpha/{code} – Fetch detailed info for selected country

Fetch data fields:

- name.common
- capital[0]
- region
- population
- languages
- flags.png

## 7. User Interface & UX

- **Framework:** Tailwind CSS for a consistent, modern design.
- **Layout:** Grid layout for country cards.
- **Navigation:** Clear, intuitive interface with persistent header.
- **Accessibility:** Semantic HTML, alt tags for flags, keyboard navigation supported.

## 8. Session Management

- **LocalStorage:** Store recent searches, filters, or favorite countries.
- **JWT-based Authentication:** Login system allowing users to mark favorite countries or preferences.

## 9. Version Control

- **Repository Hosting:** GitHub
- **Best Practices:**
  - Commits with meaningful messages
  - Using meaningful comments

## 10. Deployment Strategy

- **Platform:** Vercel (<https://explore-countries-restapi.vercel.app>)
- **CI/CD:** Automatic deployment on each commit to main branch
- **Environment Variables:** For backend
- **Custom Domain:** Configurable via Vercel DNS settings

## 11. Testing Plan

- **Unit Tests:** With Jest and React Testing
- **Manual Testing:** Browser compatibility across Chrome, Firefox, Safari
- **Functional Testing:** Search, filter, and navigation tested with different use cases
- **Responsiveness Testing:** Across screen sizes and devices

## 12. Challenges and Solutions

### 1. CORS Issues

Challenges: CORS-related errors when fetching data from REST Countries API.

Solution: Ensured the API endpoint <https://restcountries.com/v3.1/> was correctly used. Avoided unnecessary proxies by switching to their latest, CORS-compliant version.

### 2. React Router DOM Missing in Tests

Challenges: Unit test using MemoryRouter failed due to missing react-router-dom.

Solution: Installed the suitable version of package using `npm install react-router-dom` and verified test dependencies.

### 3. Netlify Build & Deployment Configuration

Challenges: Netlify failed to detect correct build paths.

Solution: Set Base directory: frontend, Build command: `npm run build`, Publish directory: build

### 4. Responsiveness Issues

Challenges: Application layout was broken on mobile viewports.

Solution: Used Tailwind CSS utility classes to apply responsive design principles

### 5. GitHub Sync Delays

Challenges: Netlify deployment didn't reflect new commits.

Solution: Verified the Netlify-GitHub connection, manually triggered builds, and ensured that changes were pushed to the correct branch.

## 13. Conclusion

The GeoNeo React application provides an engaging platform for users to explore detailed information about countries. With the integration of modern frontend technologies, RESTful data handling, and user management, the project demonstrates a comprehensive application development lifecycle from planning and coding to deployment and testing.

---

**GitHub Repository:** <https://github.com/HirushiniChandrasoma/explore-countries-restapi.git>

**Live Demo:** <https://explore-countries-restapi.vercel.app>